

Reviewer Pack — Minimal Tropical Hodge Index and Resolution Proof Width Lowe...

Entry 5afb10a4dcba · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 06:29:01 UTC

Minimal Tropical Hodge Index and Resolution Proof Width Lower Bound

Entry ID: 5afb10a4dcba **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 06:29:01 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For every satisfiable CNF formula φ with n variables, the minimal tropical Hodge index ($h_t(\varphi)$) of its associated tropical variety is linearly correlated with its resolution proof width $w(\varphi)$, such that $h_t(\varphi) = \Omega(w(\varphi))$.

Rationale (proposer's reasoning):

The tropical Hodge index captures geometric properties of the polynomial functions defining the tropical variety, which can reflect the complexity inherent in solving the CNF formula. A lower bound on this invariant could provide insights into the complexity of resolution proofs.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4c658b25be4ce915

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all CNF formulas φ with n variables ($n \leq 40$), the ratio of the minimal tropical Hodge index to the resolution proof width is greater than or equal to 1.0, and the mean ratio across all seeds is also greater than or equal to 1.0. The conjecture is falsified if there exists any seed where this ratio is less than 1.0.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "resolution proof complexity" AND minimal Hodge index" - "Boolean satisfiability" AND tropical variety AND resolution proof width" - "minimal tropical Hodge index" IN Tropical Geometry AND Boolean Satisfiability

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1001.1554v4] Tropical geometry and correspondence theorems via toric stacks - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_cnf(n):
    cnf = []
    for _ in range(10): # Generate 10 clauses
        clause = []
        for i in range(n):
            coeff = random.choice([-1, 1])
            if all(abs(coeff) != abs(cnf[j]) for j in range(len(cnf))):
                clause.append(coeff)
        cnf.append(clause)
    return cnf

def resolution_width(cnf):
    # Simplified DPLL solver to estimate resolution width
    clauses = [set(clause) for clause in cnf]
    unit_clauses = {c for c in set.union(*clauses) if len(c) == 1}

    while unit_clauses:
        unit_clause = next(iter(unit_clauses))
        unit_clauses.remove(unit_clause)

        for i, clause in enumerate(clauses):
            if unit_clause in clause:
                clauses[i] -= {unit_clause}
                if not clauses[i]:
                    return float('inf')
```

```

        elif len(clauses[i]) == 1:
            unit_clauses.add(next(iter(clauses[i])))

    return len(cnf)

def tropical_hodge_index(cnf):
    # Placeholder for actual computation
    # For simplicity, we use the number of variables as a proxy
    n = sum(len(clause) for clause in cnf)
    return Fraction(n, 2)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    ratios = []

    for n in n_values:
        cnf = generate_cnf(n)
        ht = tropical_hodge_index(cnf)
        w = resolution_width(cnf)

        if w == float('inf'):
            continue

        ratio = Fraction(ht, w)
        ratios.append(ratio)

    if not ratios:
        return {
            "metric_name": "ratio",
            "metric_value": 0.0,
            "instances_tested": 0,
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "empty_ratio"
        }

    mean_ratio = sum(ratios) / len(ratios)
    conjecture_holds = all(r >= Fraction(1, 1) for r in ratios)

    return {
        "metric_name": "ratio",
        "metric_value": float(mean_ratio),
        "instances_tested": len(ratios),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:

```

```

    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='ratio_less_than_1' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=empty_ratio")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8c8bafa1.py", line 99, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8c8bafa1.py", line 63, in run_trial
    w = resolution_width(cnf)
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8c8bafa1.py", line 32, in resolution_width
    unit_clauses = {c for c in set.union(*clauses) if len(c) == 1}
                      ~~~~~^
TypeError: object of type 'int' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's support or falsification according to the pre-registered criteria. | next: Investigate and fix the error in the test code that caused it to crash. Once the code is stable, rerun the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13498
2	preregistration	ollama_remote	glm4:latest	0	0	9835
3	novelty	ollama_remote	glm4:latest	0	0	10614
4	novelty	ollama_remote	glm4:latest	0	0	9713
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13997
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9320
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8975
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10168
9	judge	ollama_remote	glm4:latest	0	0	16296

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 102417 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5afb10a4dcba.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5afb10a4dcba.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5afb10a4dcba.tar.gz` (if generated)